Cloudflare Worker Setup Guide

Habit Tracker — Offline-First Sync Backend

June 2026

Cloudflare Worker Setup — Step-by-Step Guide

This guide walks through standing up the Cloudflare Workers + D1 backend that the Habit Tracker Android app syncs with, including the shared-secret authentication that resolves the “no API auth” security gap identified in the code review.

Part A: Account Setup (Cloudflare Webpage)

1. Create a Cloudflare account

Go to `dash.cloudflare.com`, sign up with an email address, and verify the email.

2. Get your Account ID

After logging in, go to the dashboard home page (or any domain / Workers page). The **Account ID** is shown on the right sidebar of the Workers & Pages overview page. Copy it — it may be needed in `wrangler.toml` depending on setup.

3. Enable Workers & D1

Navigate to **Workers & Pages** in the left sidebar. On first use, it may prompt for a `workers.dev` subdomain (e.g. `yourname.workers.dev`) — choose one; this becomes part of the Worker’s public URL.

No further manual setup is needed on the webpage yet — the D1 database and Worker are created via the `wrangler` CLI, and will appear automatically in the dashboard afterward.

Part B: Local Machine Setup (Terminal)

4. Install Node.js

Check with `node -v` (needs Node 18+). Install from nodejs.org if missing.

5. Install Wrangler

```
npm install -g wrangler
```

6. Authenticate Wrangler with the Cloudflare account

```
wrangler login
```

This opens a browser window — log in and click **Allow** to authorize Wrangler. This step links the terminal to the Cloudflare account created in Part A.

Part C: Create the Worker Project

7. Scaffold a new Worker project

```
npm create cloudflare@latest habit-sync-worker
```

When prompted:

- *"What would you like to start with?"* → **Hello World** (Worker only, no framework)
- *"Which language?"* → **TypeScript**
- *"Do you want to use git?"* → Yes
- *"Do you want to deploy?"* → No (deploy after setup is complete)

This creates a `habit-sync-worker` folder containing `src/index.ts` and `wrangler.toml` (or `wrangler.jsonc` in newer CLI versions).

```
cd habit-sync-worker
```

Part D: Create the D1 Database

8. Create the D1 database

```
npx wrangler d1 create habit-tracker-db
```

This outputs a block similar to:

```
[[d1_databases]]
binding = "DB"
database_name = "habit-tracker-db"
database_id = "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
```

9. Add this output to `wrangler.toml`

Paste the generated block into the Worker project's `wrangler.toml`.

10. Verify on the Cloudflare webpage

Go to **Workers & Pages → D1** in the left sidebar. `habit-tracker-db` should now be listed. Click it to confirm — this is visual confirmation the database exists in the cloud.

Part E: Create the Database Schema

11. Create a schema file locally

Create `schema.sql` in the Worker project root:

```
        CREATE TABLE IF NOT EXISTS habits (
          id TEXT PRIMARY KEY,
          name TEXT NOT NULL,
          icon TEXT,
          frequency TEXT,
          reminderTime TEXT,
          duration INTEGER,
          category TEXT,
          createdAt INTEGER,
          isDeleted INTEGER DEFAULT 0
        );

        CREATE TABLE IF NOT EXISTS habit_logs (
          id TEXT PRIMARY KEY,
          habitId TEXT NOT NULL,
          date TEXT NOT NULL,
          isDeleted INTEGER DEFAULT 0,
          UNIQUE(habitId, date)
        );

        CREATE INDEX IF NOT EXISTS idx_habit_logs_habitId ON
habit_logs(habitId);
```

12. Apply the schema to the remote (cloud) database

```
        npx wrangler d1 execute habit-tracker-db --remote --file=schema.sql
```

13. Verify on the webpage

Go to **Workers & Pages → D1 → habit-tracker-db**. Open the **Console** or **Tables** tab — `habits` and `habit_logs` should be listed with their columns. A test query such as `SELECT * FROM habits;` can be run directly in the dashboard's query console.

Part F: Add a Shared Secret for Auth

This step prepares the backend for the authentication fix (issue **H1** from the code review).

14. Generate a random secret

```
        openssl rand -hex 32
```

Copy the output — this is the API key.

15. Add it as a Worker Secret (not in code)

```
        npx wrangler secret put API_KEY
```

When prompted, paste the value generated in step 14.

16. Verify on the webpage

Go to **Workers & Pages → (the worker, after first deploy) → Settings → Variables and Secrets**. `API_KEY` should be listed as an encrypted secret with its value hidden.

Part G: Write the Worker Code

17. Replace `src/index.ts`

```
        export interface Env {
          DB: D1Database;
          API_KEY: string;
        }

        function checkAuth(request: Request, env: Env): boolean {
          const authHeader = request.headers.get("Authorization");
          return authHeader === `Bearer ${env.API_KEY}`;
        }

        function jsonResponse(data: unknown, status = 200): Response {
          return new Response(JSON.stringify(data), {
            status,
            headers: { "Content-Type": "application/json" },
          });
        }

        export default {
          async fetch(request: Request, env: Env): Promise<Response> {
            const url = new URL(request.url);

            if (!checkAuth(request, env)) {
              return jsonResponse({ error: "Unauthorized" }, 401);
            }

            if (url.pathname === "/api/habits" && request.method === "GET")
{
              const { results } = await env.DB.prepare(
                "SELECT * FROM habits WHERE isDeleted = 0"
              ).all();
              return jsonResponse(results);
            }

            if (url.pathname === "/api/sync/habits" && request.method ===
"POST") {
              const habits = await request.json() as any[];
              for (const h of habits) {
                await env.DB.prepare(
                  `INSERT INTO habits (id, name, icon, frequency,
reminderTime, duration, category, createdAt, isDeleted)
                   VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                   ON CONFLICT(id) DO UPDATE SET
                     name=excluded.name, icon=excluded.icon,
frequency=excluded.frequency,
                     reminderTime=excluded.reminderTime,
duration=excluded.duration,
                     category=excluded.category,
isDeleted=excluded.isDeleted`
                ).bind(h.id, h.name, h.icon, h.frequency, h.reminderTime,
h.duration, h.category, h.createdAt, h.isDeleted ? 1 : 0).run();
              }
              return jsonResponse({ success: true });
            }

            if (url.pathname === "/api/sync/logs" && request.method ===
"POST") {
```

```
            const logs = await request.json() as any[];
            for (const l of logs) {
              await env.DB.prepare(
                `INSERT INTO habit_logs (id, habitId, date, isDeleted)
                 VALUES (?, ?, ?, ?)
                 ON CONFLICT(id) DO UPDATE SET
isDeleted=excluded.isDeleted`
              ).bind(l.id, l.habitId, l.date, l.isDeleted ? 1 : 0).run();
            }
            return jsonResponse({ success: true });
          }

          return jsonResponse({ error: "Not found" }, 404);
        },
      };
```

Part H: Deploy

18. Deploy the Worker

```
npx wrangler deploy
```

This outputs the live URL, e.g.:

`https://habit-sync-worker.yourname.workers.dev`

19. Verify on the webpage

Go to **Workers & Pages** — `habit-sync-worker` should now be listed as a deployed Worker. Click it to view live request logs, metrics, and settings.

20. Test manually before touching the Android app

```
curl -H "Authorization: Bearer YOUR_API_KEY" https://habit-sync-
worker.yourname.workers.dev/api/habits
```

Should return `[]` (empty array, since the database is empty).

Try the same request without the header — it should return `401 Unauthorized`. This confirms authentication is working correctly.

Part I: Connect Back to Android

21. Update `local.properties` in the Android project:

```
CLOUDFLARE_BASE_URL=https://habit-sync-worker.yourname.workers.dev/
CLOUDFLARE_API_KEY=<same key generated in Part F, step 14>
```

22. Prompt for the Android AI agent

```
Add CLOUDFLARE_API_KEY as a new BuildConfig field sourced from
local.properties, same pattern as CLOUDFLARE_BASE_URL. Then add an
OkHttp Interceptor in AppModule.kt that attaches the header
"Authorization: Bearer <CLOUDFLARE_API_KEY>" to every outgoing
request. This resolves issue H1 from the earlier code review.
```

Summary Checklist

Step	Task	Verified Where
1–3	Cloudflare account + Workers enabled	Dashboard
4–6	Node, Wrangler installed & authenticated	Terminal
7	Worker project scaffolded	Terminal
8–10	D1 database created	Dashboard → D1
11–13	Schema applied	Dashboard → D1 → Console
14–16	API key secret created	Dashboard → Worker Settings
17	Worker code written	Local editor
18–20	Worker deployed & auth tested	Dashboard → Worker → Logs, `curl`
21–22	Android app updated to send auth header	Android project

Once this is complete, the backend will reject any request without the correct `Authorization` header, closing the previously identified **H1 — No API Authentication** gap, and the Android app will have a live, real backend to sync against.